Course / Section:	ICS-344 — Term 252
Student Name + ID:	s202162910
AWS Region:	us-east-1 (N. Virginia)
Date:	May 2026
Vulnerability Title:	Lesson #10 — Unhandled Exceptions in the DVSA application

Part 1) Goal and Vulnerability Summary

This lesson demonstrates an Unhandled Exception vulnerability in the DVSA application when processing the get action. The issue occurs in the backend Lambda functions (DVSA-ORDER-MANAGER and DVSA-ORDER-GET), where incomplete requests cause runtime errors that are returned directly to the client.

The main affected components are API Gateway and Lambda functions, and the impact is information disclosure. Specifically, the system leaks internal stack traces, file paths, and code details when an exception occurs. The root cause is improper input validation and lack of safe error handling, allowing backend exceptions to be exposed to the user.

Part 2) Why This Works / Root Cause

The vulnerability exists because:

- The application does not validate required input fields, such as order-id.
- The get action assumes that order-id is always provided.
- When missing, the request is still forwarded to the backend Lambda (DVSA-ORDER-GET).
- The backend function tries to access event["orderId"], causing a runtime exception.
- The main handler (DVSA-ORDER-MANAGER) returns the raw error response directly to the client without sanitization.

This leads to exposure of internal application details.

Part 3) Environment and Setup

- DVSA deployed in AWS (us-east-1)
- API endpoint:

https://<api-id>.execute-api.us-east-1.amazonaws.com/Stage/order

- Relevant components:

    API Gateway

    Lambda:

        DVSA-ORDER-MANAGER

        DVSA-ORDER-GET

    DynamoDB

- Tools used:

    - Browser DevTools (to capture JWT)

    - curl (to send requests)

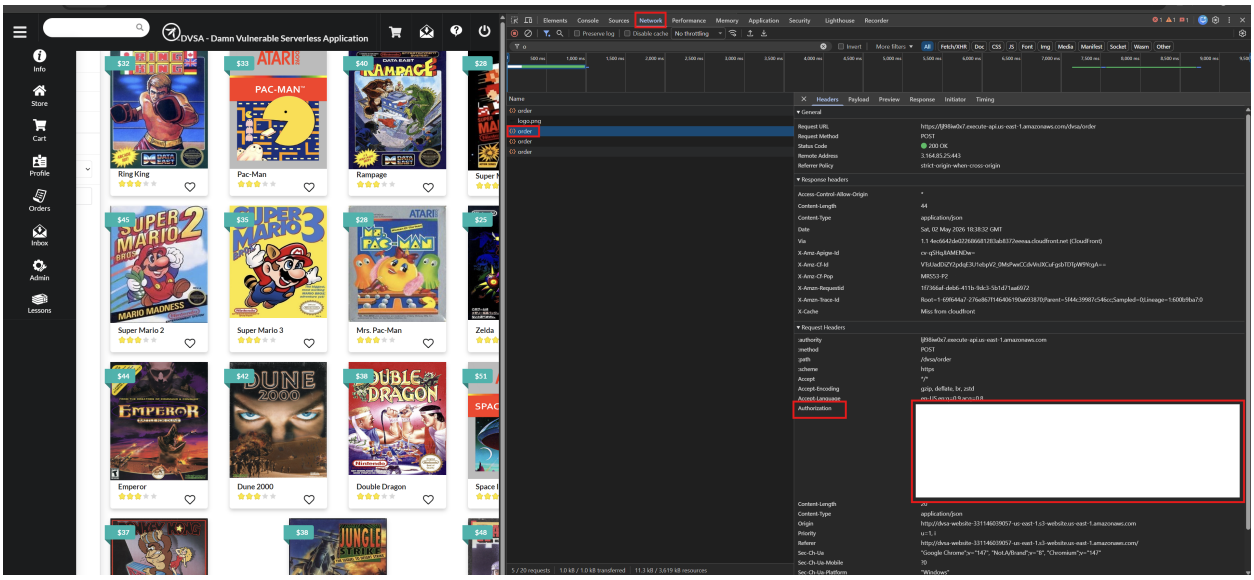
    - CloudWatch Logs (to verify backend behavior)

JWT token was obtained from the browser by inspecting the Authorization header during a normal reques

Part 4) Reproduction Steps

Log in and get JWT

1. Open DVSA in browser
2. Log in
3. Open DevTools → Network
4. Click “Orders”
5. Copy Authorization token



6. Send malformed request

curl -X POST "https://<API>/Stage/order" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <TOKEN>" \
-d '{"action":"get"}'

Observe response

The response contains:

- stack trace
- error type (KeyError)
- internal file path (/var/task/...)

Part 5) Evidence and Proof

```
(base) turki@DESKTOP-GKGPS4R:~/344Proj$
curl -X POST "https://ljl98iw0x7.execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer
```

```
-d '{"action":"get"}'
}'
{"errorMessage":"'orderId'","errorType":"KeyError","stackTrace":["  File \"/var/task/get_order.py\", line 30, in lambda_handler\n    orderId = event[\"orderId\"]\n"]}> ^C
```

The response body shows stack trace entries

Part 6) Fix Strategy / Probable Mitigation

The vulnerability can be fixed by:

1. Input validation
    - Ensure order-id exists before processing
    - Reject malformed requests early
2. Exception handling
    - Catch runtime errors in Lambda functions
    - prevent propagation of raw exceptions
3. Safe responses
    - Return generic error messages
    - keep detailed logs in CloudWatch only

Part 7) Code / Config Changes

DVSA-ORDER-MANAGER

Before:

```
59              case "get":
60                  payload = { "user": user, "orderId": req["order-id"], "isAdmin": isAdmin };
61                  functionName = "DVSA-ORDER-GET";
62                  break;
```

After (input validation)

```
59              case "get":
60                  if (!req["order-id"] || typeof req["order-id"] !== "string") {
61                      const response = {
62                          statusCode: 400,
63                          headers: {
64                              "Access-Control-Allow-Origin": "*"
65                          },
66                          body: JSON.stringify({
67                              status: "err",
68                              message: "Missing or invalid order-id"
69                          })
70                      };
71                      callback(null, response);
72                      return;
73                  }
74                  payload = { "user": user, "orderId": req["order-id"], "isAdmin": isAdmin };
75                  functionName = "DVSA-ORDER-GET";
76                  break;
77
```

DVSA-ORDER-GET

Before:

```
orderId = event["orderId"]
userId = event["user"]
is_admin = json.loads(event.get("isAdmin", "false").lower())
address = "{}"

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table( os.environ["ORDERS_TABLE"] )

if is_admin:
    response = table.query(
        KeyConditionExpression=Key('orderId').eq(orderId)
    ).get("Items", [None])

else:
    key = {"orderId": orderId, "userId": userId}
    response = [table.get_item(Key=key).get("Item")]

res = {"status": "ok", "order": response[0] } if response[0] is not None else { "status": "err", "msg": "could not find order" }

return json.loads(json.dumps(res, cls=DecimalEncoder).replace("\\\"", "\"").replace("\\n", ""))
```

After (Exception handling with appropriate response):

```
29      try:
30          orderId = event["orderId"]
31          userId = event["user"]
32          is_admin = json.loads(event.get("isAdmin", "false").lower())
33          address = "{}"
34
35          dynamodb = boto3.resource('dynamodb')
36          table = dynamodb.Table( os.environ["ORDERS_TABLE"] )
37
38          if is_admin:
39              response = table.query(
40                  KeyConditionExpression=Key('orderId').eq(orderId)
41              ).get("Items", [None])
42
43          else:
44              key = {"orderId": orderId, "userId": userId}
45              response = [table.get_item(Key=key).get("Item")]
46
47          res = {"status": "ok", "order": response[0] } if response[0] is not None else { "status": "err", "msg": "could not find order" }
48
49          return json.loads(json.dumps(res, cls=DecimalEncoder).replace("\\\"", "\"").replace("\\n", ""))
50      except Exception as e:
51          print(str(e))
52          return {"status": "err", "msg": "Internal server error"}
```

Part 8) Verification After Fix

Repeat the same request:

The system ensures secure error handling by suppressing stack traces and preventing internal information leakage, while valid requests continue to function normally.

Part 9) Structured Operation and Security Analysis

1) Intended Logic and Security Rule(s)

The get operation should allow an authenticated user to retrieve an order by providing a valid order-id. The request flows from API Gateway to DVSA-ORDER-MANAGER, then to DVSA-ORDER-GET, which queries DynamoDB. The system should validate required inputs and return only safe error messages. Internal details such as stack traces must not be exposed.

2) Evidence Sources and Behavior Trace

Analysis was based on API responses, Lambda code, and logs. Normally, a valid request returns order data. In the exploit case, sending {"action":"get"} without order-id causes a backend exception and returns a stack trace. After the fix, the same request returns a safe error (e.g., Missing orderId) with no leakage.

3) Deviation Analysis and Classification

The system fails to validate input and exposes backend errors instead of handling them safely. This violates the rule that internal exceptions must not be returned to the client. This is classified as intentional misuse / security-relevant abuse.

4) Explainable Fix and Post-Fix Validation

The issue was caused by the incorrect assumption that the get request would always include a valid order-id. The backend logic directly accessed this field without validation, which led to runtime exceptions when the input was missing or malformed. The fix belongs in the backend request-handling path, specifically in the DVSA-ORDER-GET Lambda (and optionally at the DVSA-ORDER-MANAGER level).

The exact change made was to introduce input validation and/or exception handling so that missing or invalid orderId is handled safely instead of causing a crash. This ensures that the backend no longer throws unhandled exceptions for malformed requests.

Post-fix validation confirms the issue is resolved because the same malformed request no longer returns stack traces or internal details. Instead, the application returns a safe error message. Additionally, legitimate behavior still works correctly, as valid requests with a proper order-id continue to return the expected

Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #10: Unhandled Exceptions (get)	The get request must include a valid order-id; invalid input must be rejected safely; backend errors must not be exposed to the client	API responses, Lambda code (DVSA-ORDER-MANAGER, DVSA-ORDER-GET), CloudWatch logs, DevTools request capture	A valid request with order-id returns the correct order or a safe error message if not found	Sending {"action":"get"} (missing order-id) causes a backend exception and returns stack trace, file paths, and error details

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson #10: Unhandled Exceptions (get)	The system fails to validate input and returns internal exception details instead of safe messages,	Intentional misuse / security-relevant abuse	Input validation and exception handling added in DVSA-ORDER-GET (and/or DVSA-	After fix, the same request returns a safe message (e.g., Missing orderId) and no stack trace, while	N/A

	violating the intended rule of secure error handling		ORDER-MANAGER)	valid requests still work	